

Modelling Room Acoustics on a GPU

A new method that improves memory efficiency

Jacob Josiah Webber

Dissertation Presentation

Introduction

This talk aims to answer the following questions:

- ▶ Modelling acoustics: what and why?
- ▶ What is the current state-of-the-art?
- ▶ What is the new method proposed here?
- ▶ What were the results when this method was used?
- ▶ How could these be improved?

Sound

What is it?

From Wikipedia

“In physics, sound is a vibration that propagates as a typically audible mechanical wave of pressure and displacement, through a transmission medium such as air or water.”

From my undergraduate degree

$$\nabla^2 P - \frac{1}{c^2} \frac{\partial^2 P}{\partial t^2} = 0 \quad (1)$$

Room acoustics

- ▶ The physical properties of rooms affects how we hear sound.
- ▶ Sound reverberates off walls/objects.
- ▶ Waves diffract around corners.
- ▶ Some rooms echo, some sound dry.
- ▶ Backwards: If we know physical properties can we derive the sound?

The wave equation

$$\frac{\partial^2 P}{\partial x^2} - \frac{1}{c^2} \frac{\partial^2 P}{\partial t^2} = 0$$

where P is pressure deviation and c is speed of sound in medium.

- ▶ Very familiar to those who have studied physics
- ▶ It, and its application to acoustics, are derivable from first principles.
- ▶ Solving for P gives us acoustic properties of a system (boundary conditions are very important!).
- ▶ Solving this analytically for complex systems is not possible.

FTDT

Finite-difference time-domain method

Finite difference methods are a way of approximating and discretising differential equations so that they can be solved numerically.

Start off by replacing derivative with finite difference:

$$\frac{df(u)}{du} \approx \frac{f(u+h) - f(u-h)}{2h} \quad (2)$$

$$\frac{d^2f(u)}{du^2} \approx \frac{f(u+h) - 2f(u) + f(u-h)}{h^2} \quad (3)$$

Let h be the distance between samples (H) and P be a 3D grid

$$\delta_t^2 P_{l,m,p}^n \equiv \frac{1}{T^2} \left(P_{l,m,p}^{n+1} - 2P_{l,m,p}^n + P_{l,m,p}^{n-1} \right) \approx \frac{\partial^2 P}{\partial t^2} \quad (4)$$

$$\delta_x^2 P_{l,m,p}^n \equiv \frac{1}{H^2} \left(P_{l+1,m,p}^n - 2P_{l,m,p}^n + P_{l-1,m,p}^n \right) \approx \frac{\partial^2 P}{\partial x^2} \quad (5)$$

$$\delta_y^2 P_{l,m,p}^n \equiv \frac{1}{H^2} \left(P_{l,m+1,p}^n - 2P_{l,m,p}^n + P_{l,m-1,p}^n \right) \approx \frac{\partial^2 P}{\partial y^2} \quad (6)$$

$$\delta_z^2 P_{l,m,p}^n \equiv \frac{1}{H^2} \left(P_{l,m,p+1}^n - 2P_{l,m,p}^n + P_{l,m,p-1}^n \right) \approx \frac{\partial^2 P}{\partial z^2} \quad (7)$$

$$\frac{\partial^2 P}{\partial t^2} = c^2 \left(\frac{\partial^2 P}{\partial x^2} + \frac{\partial^2 P}{\partial y^2} + \frac{\partial^2 P}{\partial z^2} \right) \quad (8)$$

$$P_{l,m,p}^{n+1} - 2P_{l,m,p}^n + P_{l,m,p}^{n-1} = \frac{c^2 T^2}{H^2} \left(P_{l+1,m,p}^n + P_{l-1,m,p}^n + P_{l,m+1,p}^n + P_{l,m-1,p}^n \right. \\ \left. + P_{l,m,p+1}^n + P_{l,m,p-1}^n - 6P_{l,m,p}^n \right) \quad (9)$$

Letting the Courant number be defined as

$$\lambda \equiv \frac{cT}{H} \quad (10)$$

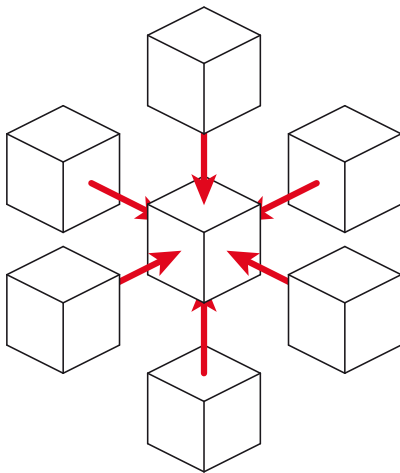
and our stencil

$$S \equiv P_{l+1,m,p}^n + P_{l-1,m,p}^n + P_{l,m+1,p}^n + P_{l,m-1,p}^n + P_{l,m,p+1}^n + P_{l,m,p-1}^n \quad (11)$$

gives a somewhat neater version of equation 9

$$P_{l,m,p}^{n+1} = (2 - 6\lambda^2)P_{l,m,p}^n + \lambda^2 S - P_{l,m,p}^{n-1} \quad (12)$$

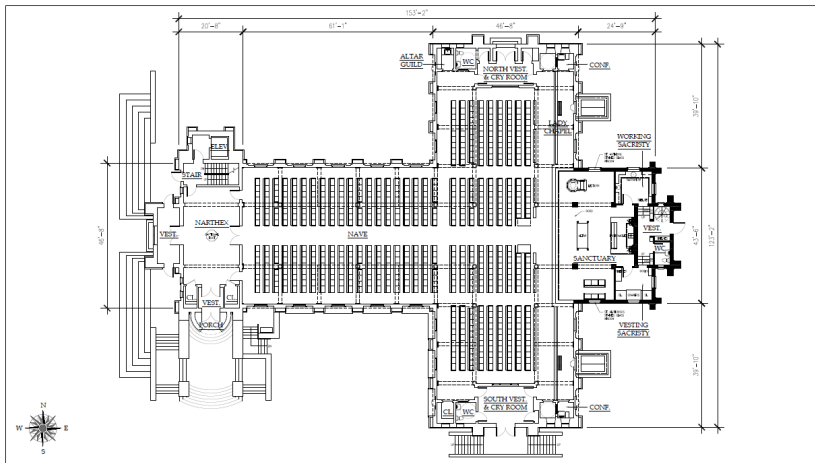
Stencil



Memory considerations

For stencilling operations, memory access is the bottleneck.

- ▶ Accurate models need a lot of points to model a room.
- ▶ Wavelength of sound at 20kHz is 17mm.
- ▶ High memory bandwidth means GPUs are used - limited memory.
- ▶ Standard 3d arrays are cuboid.
- ▶ Due to memory layout, neighbouring points in space will always be far apart in memory.



McCREY Architects
100 Main Street, Suite 100
Rumson, NJ 07070
(201) 717-1144 www.mccreyarchitects.com

HOLY CROSS CHURCH RUMSON, NEW JERSEY

Main Floor Plan
1/16" = 1'-0"
December 29, 2011
Sheet 4 of 21

The Proposal

Use an array of structs instead!

- ▶ Store enough points for CUDA kernel to operate on.
- ▶ Only store structs with at least one point inside the room.
- ▶ Experiment with different sized arrays within the struct to see if caching benefits can be yielded.

The Struct

Listing 1: The block struct

```
typedef real bl_array[Bz][By][Bx];  
struct block {  
    int up;  
    int down;  
    int left;  
    int right;  
    int fore;  
    int aft;  
    bl_array u;  
    bl_array u1;  
    char k[Bz][By][Bx];  
};
```

Converting between representations

Going from bounding box to array of structs

- ▶ Implement struct in 2d, then 3d
- ▶ Implement algorithm to convert from bounded array to array of structs.
- ▶ See if this uses less memory (it does!)
- ▶ See if this method is intolerably slow in CUDA.
- ▶ See if results are **correct**.
- ▶ Experiment with different sub-array sizes to optimise for
 - ▶ memory usage
 - ▶ caching efficiency

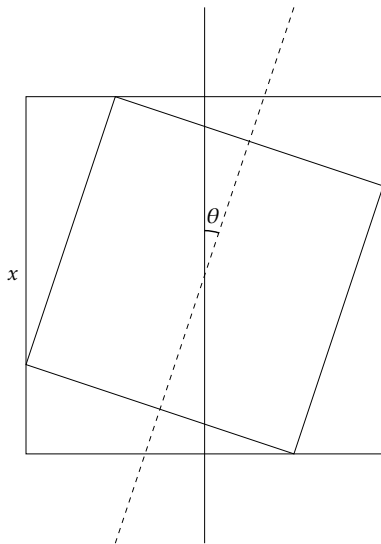


Figure: Diagram showing unit square rotated by θ within bounding square, where x is the side of the bounding square.

The End

This talk aimed to answer the following questions:

- ▶ What is sound?
- ▶ How do the spaces we hear them in affect this sound?
- ▶ What methods currently exist to simulate this?
- ▶ Why would we want to simulate this?
- ▶ How will I try to improve these?